BACKEND REQUIREMENTS

Storage Rack

Models, Data Operations, API, Pages & Process Flows — Extracted from Frontend

Source: D:\storage_rack — a static HTML/vanilla-JS application with no real backend today.

js/common.js currently serves as the "database" via browser localStorage.

This document specifies the exact data models, required operations, REST API, per-page call sequence, and step-by-step process flows needed to build a real backend for this app. No implementation code — requirements only.

Generated 2026-10-15

Backend Folder Structure

Where the models, serializers, permission checks and API views specified in the sections below actually live, inside the project root (`D:\storage_rack\backend\`). The backend is split across **two Django apps**: `accounts` owns the Users and Roles pages (`Role` , `User` , login/invite, and the shared permission check), and `storage_rack` owns everything else (Locations, Storage Units, Items, IN/OUT Log). Each app also gets its own `services.py` — the business-logic layer multi-step operations (invite/reset-password, archive/restore/move, in-use counts) live in, separate from the thin HTTP-handling `views.py` . The only cross-app coupling is `storage_rack/views.py` importing `HasModulePermission` from `accounts.permissions` , since permission checking is inherently about `Role` .

> This redesigns a backend that was previously scaffolded and verified end to end as a single app (`pip install -r requirements.txt` , `manage.py check` , `makemigrations` , `migrate` against a real SQLite database, dev server boots cleanly). Splitting an already-working single app into two is a mechanical, low-risk refactor — move files, split `INSTALLED_APPS` , add the one cross-app import above — rather than new behavior, but the two-app layout below has not itself been re-scaffolded and re-run on disk; treat it as a documented, internally-consistent proposal, not a freshly re-confirmed one. One change was made beyond the plain field-mirror in Section 1: `User` extends `django.contrib.auth.models.AbstractUser` instead of a bare model, with `AUTH_USER_MODEL = "accounts.User"` set in `settings.py` — required for a real login endpoint to work at all, not optional.

```
backend/
├── manage.py
├── requirements.txt              Django, djangorestframework, django-cors-headers
├── .gitignore                    excludes __pycache__/, db.sqlite3, .env, venv/
├── db.sqlite3                    created by "migrate" — gitignored, not committed
├── backend/                      — project config (from "django-admin startproject backend")
│   ├── __init__.py
│   ├── settings.py                INSTALLED_APPS, MIDDLEWARE, REST_FRAMEWORK, AUTH_USER_MODEL, CORS_ALLOWED_ORIGINS
│   ├── urls.py                    mounts accounts.urls and storage_rack.urls, both under "api/"
│   ├── wsgi.py
│   └── asgi.py
├── accounts/                     — Users & Roles app (from "python manage.py startapp accounts")
│   ├── __init__.py
│   ├── models.py                  Role, User (extends AbstractUser) — see Section 1
│   ├── serializers.py             RoleSerializer, UserSerializer, camelCase field mapping — see Section 3 (API Reference)
│   ├── permissions.py             HasModulePermission — checks role.permissions[module][action] — imported by storage_rack too —
see Section 6
│   ├── services.py                invite / reset-password logic, role in-use count
│   ├── views.py                   RoleViewSet, UserViewSet + invite action — see Sections 2 & 3
│   ├── auth_views.py              LoginView, LogoutView — replaces the hardcoded demo credential — see Section 6
│   ├── urls.py                    DefaultRouter for roles/users + auth routes — produces the Users/Roles/auth endpoints in
Section 3
│   ├── admin.py                   registers Role and User with the Django admin site
│   ├── apps.py
│   └── migrations/
│       ├── __init__.py
│       └── 0001_initial.py          generated by "python manage.py makemigrations accounts"
└── storage_rack/                 — Storages, Items, Locations & IN/OUT Log app (from "python manage.py startapp storage_rack")
    ├── __init__.py
    ├── models.py                  Location, StorageUnit, Item, ItemLogEntry — see Section 1
    ├── serializers.py             one DRF serializer per model, camelCase field mapping — see Section 3 (API Reference)
    ├── services.py                archive/restore/move logic, location in-use count
    ├── views.py                   one ModelViewSet per resource, + archive/restore/move actions — see Sections 2 & 3
    ├── urls.py                    DefaultRouter for locations/storages/items/log — produces the remaining endpoints in Section 3
    ├── admin.py                   registers the remaining four models with the Django admin site
    ├── apps.py
    └── migrations/
        ├── __init__.py
        └── 0001_initial.py          generated by "python manage.py makemigrations storage_rack"
```

Modules (Pages)

Every page in the app, and — in backend-requirements terms — what it actually needs once a real server exists. This is a quick index; the exact endpoint sequence for each page is in Section 4 (Pages → API Call Map), and the step-by-step behavior behind any multi-step action is in Section 5 (Key Process Flows). Both views (`/web` , `/mobile`) ship their own copy of every page below except Scan QR, which is mobile-only — the split is presentation-only and has no effect on what the backend needs to provide.

View Chooser — `/index.html`
Needs nothing from the backend at all — it's a static Mobile/Web picker shown before any login, with no data of its own.

Login — `index.html`
Needs `POST /api/auth/login` and the Forgot Password pair (Section 3.1). This is the one page where the current frontend's behavior (a single hardcoded credential, plus a direct `srUsersV1` check) must **not** be carried over as-is — see Section 6 for what a real implementation requires instead.

Overview (Dashboard) — `dashboard.html`
Needs the three read-only endpoints in Section 3.8 — a summary count plus two reuses of the Storage Units list endpoint with different sort/filter params. No writes happen on this page.

Items — `items.html`
The heaviest page: full CRUD on Section 3.4, plus `/archive` , `/restore` , `/move` , `/reassign` , and `/import` . Its Type filter needs the backend to resolve a storage's `customType` label and to support the special `unassigned` value (Section 1, Section 3.4) — neither is a new endpoint, just filtering behavior the existing `GET /api/items` must support.

Storages — `storage.html`
Full CRUD on Section 3.3. Creating a storage can optionally reassign a batch of existing items in the same request (`existingItemIds`) — see the Add Storage flow in Section 5. Deleting one is the integrity gap called out in Section 7: today nothing checks or reassigns its attached items first.

Storage Detail — `storage-detail.html`
Reads one Storage Unit plus its Items (`storageId=` filter on Section 3.4), then issues the same per-field `PATCH` / item endpoints as the Items and Storages pages — no new endpoints of its own, just a focused view over existing ones.

Item Detail — `item-detail.html`
Reads one Item plus its Activity (`itemName=` filter on Section 3.7), then issues the same per-field `PATCH` / archive / restore / move endpoints as the Items page. Also no new endpoints — a focused view over Section 3.4 and 3.7.

Scan QR — `mobile/scan.html` (mobile only)
Needs exactly one existing endpoint: `GET /api/storage-units/{id}` , to confirm a scanned or typed ID is real before redirecting to its detail page (Section 5). Camera access and QR decoding happen entirely on-device — no backend involvement at all beyond that one lookup.

IN/OUT Log — `log.html`

Read-only: `GET /api/item-log` (Section 3.7). Never written to directly by any client call — every row is a side effect of an Item operation elsewhere (Section 2, Section 7).

Locations — `storerooms.html`

Full CRUD on Section 3.2. Delete must be refused while any storage unit still belongs to it (Section 7) — the frontend's own pre-check is a convenience, not a substitute for the backend enforcing it too.

Users — `users.html`

Full CRUD on Section 3.5, plus `/invite`. Email is required on every create/update since it's the only channel the invite flow has to reach the user (Section 1, Section 5).

Roles & Permissions — `roles.html`

Full CRUD on Section 3.6. Delete must be refused while any user still holds the role (Section 7) — this is also what Section 6's permission model is actually built from.

Services (Shared Business Logic)

The frontend's entire data layer and business logic lives in one shared file, `js/common.js`, called into by every page. Each function below encodes a rule that either needs a direct backend equivalent, or is purely client-side UI with nothing for the backend to replicate — knowing which is which matters here, since building server-side versions of the UI helpers would be wasted effort.

Needs a backend equivalent

loadRacks / loadGlobalItems / loadRooms / loadUsers / loadRoles / loadLog — replaced wholesale by the `GET` list endpoint for that resource (Section 3); **saveRacks / saveGlobalItems / saveRooms / saveUsers / saveRoles** — replaced by the targeted `POST` / `PUT` / `PATCH` / `DELETE` call each mutation actually needs, instead of rewriting the whole collection on every change the way `localStorage` does today.

findRack / findGlobalItem / findRoom / findRole / findUser — become ordinary `GET .../{id}` lookups (Section 3); a 404 must be distinguishable from "deleted" the same way the frontend's own lookups already are.

nextRoomId / nextUserId / nextRoleId — Location/User/Role IDs are server-generated once there's a real database (auto-increment or similar); **Storage Unit and Item IDs stay client-supplied** (Section 1, Section 2) — `nextItemCode` only covers the case where the client leaves it blank.

archiveGlobalItem / restoreGlobalItem / purgeGlobalItem — Section 3.4's `/archive`, `/restore`, and `DELETE` respectively; each one's exact side effects (what gets logged, what doesn't) are in Section 5's Archive/Restore/Permanently Delete flow.

logTransaction / logStatusChange — the append-only writes behind every `ItemLogEntry` row; never called directly by a client, always a side effect of an Item mutation (Section 2, Section 7).

deleteRack — `DELETE /api/storage-units/{id}`, including the integrity decision Section 7 flags as still open (block vs. auto-unassign attached items).

permissionSummary / the Roles & Permissions matrix itself — this is exactly what Section 6's `HasModulePermission`-style server-side check has to enforce on every request; today it's UI-only and enforces nothing.

The Move / Use Item quantity-split logic, the Add Item "existing vs. new item" branch, and the Bulk Import row-by-row validation — these are not simple field functions but full flows; see Section 5 for the exact step order and branching each one needs reproduced server-side.

Purely client-side — no backend equivalent needed

confirmDialog, showToast / setFlashMessage — UI feedback only; nothing for the backend to implement.

openAddItemModal / openEditModal / openMoveItemModal / openQuickAddStorageModal / openQuickAddLocationModal / openLightbox, createSearchableSelect — modal and combobox UI construction; the backend only ever sees the API call these modals eventually make, never the modal itself.

buildRackCard / buildRackRow / buildItemsPreviewHtml / tagsHtml / categoryBadgeHtml / typeBadgeHtml / itemStatusBadgeHtml / imagesPreviewHtml / rackQrThumbHtml, createTagInput / createImageInput, populateRackSelect / roomOptionsHtml, renderPagination / paginateArray, animateNumber / escapeHtml / svgIcon, getCategoryColor / getItemColor / hashString — rendering, pagination, and cosmetic helpers (including the "Unassigned" status-badge override and the searchable Type-filter combobox described in Section 1's data notes) that operate entirely on data the backend already returned; none of them imply any new endpoint or field.

1. Entities (Data Models)

Six distinct data entities were identified, each currently stored as its own key in browser localStorage. Field names and types below are exactly as used in the existing code — no field has been added, dropped, or renamed from what the frontend actually reads and writes.

StorageUnit

`srStorageV4`

`id, name` `string` e.g. id = "ST-1008"

`storeRoomId` `string` FK → Location

`type` `enum` rack | cupboard | bureau | wardrobe | shelf | cabinet | box | custom

`customType` `string` free-text label the user supplies when type is "custom" (e.g. "Safe Box"); shown everywhere this storage's type is displayed instead of the generic word "Custom"; empty for every built-in type

`tags` `string[]` free-form category tags

`images` `string[]` base64 data-URI pictures

`nodes` `array` legacy sub-structure tree — deprecated, storages are flat and items attach directly today

`createdAt` `number` epoch ms

Deleting a storage unit today does not reassign or block on its attached items — see Section 7. The id is client-supplied at creation time (the "Add Storage" form has a required Storage ID field) — it is not server-generated. customType is required on create whenever type is "custom", and should stay validated that way server-side too.

Item

`srGlobalItemsV1`

`id, name` `string` e.g. id = "IT-1013"

`tags` `string[]` category tags

`quantity` `number` units on hand

`remarks` `string` free-text notes

`storageId` `string | null` FK → StorageUnit.id (null = unassigned)

`images` `string[]` base64 data-URI pictures

`status` `enum` in_store | in_use | removed (archived)

`createdAt` `number` epoch ms

id is optional on create — the Add Item form generates one if left blank, but accepts a client-supplied id too. Moving a partial quantity to another storage/status splits the row into two Items — see Section 5's Move/Use Item flow. The Items table shows "Unassigned" in its Status column whenever `storageId` is null — that is a display-only override of the actual `status` value (which is untouched and still one of the three values above); it is not a fourth status value and needs no backend change beyond the existing `storageId` null semantics.

Location

`srRoomsV1`

`id, name, location` `string` e.g. id = "SR-1002"; location = physical area/warehouse

`createdAt` `number` epoch ms

User

`srUsersV1`

`id, name, username, email` `string` e.g. id = "US-1002"; email is required — it's the only channel the invite/password-reset flow has to address the user

`phone` `string` optional

`roleId` `string` FK → Role

`status` `enum` active | inactive — inactive users are rejected at login even with a correct password

`password` `string | undefined` plaintext demo value, undefined until set; not settable from the Users page UI itself (see Section 5's Invite / Reset Password flow) — only usable once something populates it

`createdAt` `number` epoch ms

login.js now checks this field directly, in addition to the single hardcoded admin/admin123 credential: any user with a password set and status: "active" can sign in with their own username + password — see Section 5 and Section 6.

Role

`srRolesV1`

`id, name, description` `string` e.g. id = "RL-1002"

`permissions` `object` per module: { create, read, update, delete }

`createdAt` `number` epoch ms

Permission modules: storage, items, rooms, users, roles. There is no isSystem flag and no allProjects-style scoping override — every permission check is a flat per-module CRUD gate (see Section 6).

ItemLogEntry (append-only)

`srItemLogV1`

`id` `string` unique log entry ID

`timestamp` `number` epoch ms

`type` `enum` in | out | in_store | in_use | removed

`itemName` `string` denormalized snapshot of the item's name at the time of the entry

`quantity` `number` units moved

`rackId / rackName` `string` storage unit involved (denormalized); rackId may be "" when the item is unassigned

`path` `string | null` legacy sub-structure path — every call site in the current codebase passes null, so this field is effectively dead today but still present on every row

`user` `string` acting user's username (from sessionStorage) — not a foreign key to User.id

Entries are never created directly by a client call — they are always a side effect of an Item mutation (see Section 2 and the flows in Section 5).

2. Required Data Operations (by Entity)

What the backend must be able to do with each entity, stated as requirements rather than implementation. Every item below corresponds to something a page or form in the app already does today against localStorage.

StorageUnit

List — searchable by name, ID, or tag; filterable by type and location; sortable by name, date added, location, type, or item count; paginated.

Get one — by ID. A missing ID must be distinguishable from "deleted" so the UI can say so.

Create — requires a client-supplied unique ID, a name, a location, and a type; tags and pictures are optional. May optionally take a list of existing Item IDs to reassign into this new storage at the same time it's created.

Update — any of name, location, tags, or pictures, independently or together — the app saves each field separately when a user edits it inline, so partial updates must be supported.

Delete — see the Integrity Requirements in Section 7; the current app has no safeguard for storage units that still have items in them, which the backend needs to decide on.

Item count (derived) — total quantity of active (non-removed) items currently in a storage unit; needed everywhere a storage is listed or shown, not just on its own detail page.

Item

List — searchable by name, item ID, storage ID/name, tags, or remarks; filterable by storage type, status, and a specific storage; an "archived" view that shows only removed items; paginated.

Get one — by ID.

Create — name and quantity are required; storage, tags, remarks, and pictures are optional; ID may be supplied or generated; status always starts as "in store"; must record an IN movement.

Update — any of name, tags, quantity, remarks, storage, or pictures, independently or together; a quantity change must record a movement (see Section 5).

Archive — marks an item removed/inactive and records the change; does not delete the record.

Restore — reverses an archive, bringing the item back to an active "in store" status, and records the change.

Permanently delete ("purge") — only allowed once an item has already been archived; removes the record entirely and is not logged.

Move / use — the most involved operation on this entity; see the full flow in Section 5.

Reassign to a different storage — a lighter-weight version of Move that only changes which storage an item belongs to, used when a user is adding a new item but instead picks an existing one to relocate; see Section 5.

Bulk import — accepts a spreadsheet/CSV of rows, validates each independently, creates the valid ones, and reports which rows were skipped and why; see Section 5.

Location

List — searchable by name or area text; each row needs a count of how many storage units belong to it.

Get one / Create / Update — name and area text.

Delete — must be refused while any storage unit still belongs to this location.

User

List — searchable by name, username, or email; filterable by role.

Get one / Create — name, username (must be unique, case-insensitively), email, status, and a role assignment.

Update — any of the above fields.

Delete — currently unconditional in the app (no safeguards) — see Section 7 for why the backend should probably do better.

Role

List — searchable by name or description; each row needs a count of how many users currently hold this role.

Get one / Create / Update — name, description, and a full create/read/update/delete permission flag for each module (storage, items, locations, users, roles).

Delete — must be refused while any user still holds this role.

ItemLogEntry

List — searchable by item name, storage, or acting user; filterable by movement type, storage, and a date range; paginated.

No create, update, or delete operation is ever called directly — every row is written automatically as a side effect of an Item operation (create, update, archive, restore, move). This entity is read-only from the outside.

3. REST API Reference

Every endpoint the frontend implies, grouped by resource, with the exact request/response shape and the permission each one requires. This app has no network API today — every endpoint below is currently a plain in-memory array mutation followed by a localStorage write.

3.1 Auth

POST `/api/auth/login`

Authenticates and starts a session. Currently a stub (single hardcoded username/password) — see Section 6 for what must change.

REQUEST

```
{ "username": "admin", "password": "admin123" }
```

RESPONSE

```
{ "token": "...", "user": { "id": "US-1001", "name":
"Admin",
  "username": "admin", "roleId": "RL-1001" } }
```

POST `/api/auth/logout`

Invalidates the current session/token.

POST `/api/auth/forgot-password/request`

Starts the reset flow by emailing a verification code. See the full flow in Section 5.

REQUEST

```
{ "username": "admin" }
```

RESPONSE

```
{ "message": "Code sent to ad**@storage.app" }
```

> **Security requirement:** the current demo returns the generated code directly in the UI response (since there is no real email service). A real backend must **never** include the code in the API response — it must only be delivered via the actual email.

POST `/api/auth/forgot-password/confirm`

Completes the reset. Mirrors the exact client-side validation: code must match, new password ≥ 6 chars, confirmation must match.

```
{ "username": "admin", "code": "482913", "newPassword": "...", "confirmPassword": "..." }
```

3.2 Locations

GET `/api/locations?search=`

requires `rooms.read`

```
[ { "id": "SR-1002", "name": "Main Warehouse", "location": "Ground Floor",
    "createdAt": 1733900000000, "unitCount": 4 } ]
```

POST `/api/locations`

requires `rooms.create`

```
// request: { "name": "Main Warehouse", "location": "Ground Floor" }
// response: Location
```

PATCH `/api/locations/{id}`

requires `rooms.update`

```
// request: { "name": "...", "location": "..." }  (both required)
// response: Location
```

DELETE `/api/locations/{id}`

requires `rooms.delete`

409 Conflict if any storage unit still belongs to it, with the count in the error body so the client can show "Move or delete its N storage unit(s) first".

3.3 Storage Units

GET `/api/storage-units?search=&type=&roomId=&sort=&page=`

requires `storage.read`

`sort` one of: name-asc, name-desc, createdAt-desc (Newest First), createdAt-asc (Oldest First), roomLabel-asc, typeLabel-asc, itemCount-asc, itemCount-desc. `type` is one of the built-in type keys, or a case-insensitive match against any storage's `customType` label (see Section 1) — the frontend's Type filter surfaces every distinct custom label actually in use as its own searchable option, not just a generic "Custom" bucket.

```
{ "results": [ { "id": "ST-1008", "name": "Chem Storage", "storeRoomId": "SR-1002",
    "type": "cupboard", "customType": "", "tags": ["Lab"], "images": [], "createdAt": 1733900000000,
    "itemCount": 14 } ], "total": 23, "page": 1, "pageSize": 6 }
```

GET `/api/storage-units/{id}`

requires `storage.read`

Returns 404 with a body the client renders as "Storage ... was not found. It may have been deleted." — exact text used today.

POST `/api/storage-units`

requires `storage.create`

```
{ "id": "ST-1050", "name": "Rack A1", "storeRoomId": "SR-1002", "type": "rack",
  "customType": "", "tags": ["Electronics"], "images": [], "existingItemIds": ["IT-1013", "IT-1020"] }
// id is required and client-chosen; 409 if it already exists (case-insensitive)
// customType is required (non-empty) when type is "custom"; otherwise ignored/blank
// existingItemIds is optional — see the Add Storage flow in Section 5
```

PATCH `/api/storage-units/{id}`

requires `storage.update`

Accepts any subset of `{name, storeRoomId, tags, images}` — backs both the full Edit form and the individual click-to-edit fields on the storage detail page. `type` / `customType` are not in this list: the current UI never edits a storage's type after creation, only shows it read-only.

DELETE `/api/storage-units/{id}`

requires `storage.delete`

See the Integrity Requirements in Section 7 — the current app does not check or reassign attached items before deleting.

3.4 Items

GET `/api/items?search=&type=&status=&storageId=&archived=&sort=&page=`

requires `items.read`

`type` is the Storage Unit's own type, matched through `storageId` (items have no `type` field of their own — see Section 1) — one of the built-in type keys, or a case-insensitive match against a specific storage's `customType` label (see the Storage Unit entity's `customType` field above), or the special value `unassigned`, which the backend should treat as "`storageId IS NULL`" rather than trying to match it against any storage's type. `sort` one of: name-asc, name-desc, createdAt-desc (Newest First), createdAt-asc (Oldest First), storageLabel-asc (Storage A-Z), typeLabel-asc (Type A-Z), quantity-desc (most), quantity-asc (fewest).


```
{ "results": [ { "id": "IT-1013", "name": "Beaker Set", "tags": ["Lab"], "quantity": 6,
    "remarks": "", "storageId": "ST-1008", "images": [], "status": "in_store",
    "createdAt": 1733900000000 } ], "total": 112, "page": 1, "pageSize": 6 }
```


GET `/api/items/{id}`

requires `items.read`

POST `/api/items`

requires `items.create`

```
{ "id": "IT-1099", "name": "Wrenches", "tags": ["Tools"], "quantity": 16,
  "remarks": "Reorder when below 5", "storageId": "ST-1001", "images": [] }
// id optional — server generates one if omitted; 409 if a supplied id already exists
// status always starts "in_store"; records an IN movement — see Section 5
```

PATCH `/api/items/{id}`

requires `items.update`

Accepts any subset of `{name, tags, quantity, remarks, storageId, images}`. A quantity change must record a movement — see Section 5.

POST `/api/items/{id}/archive`

requires `items.delete`

Marks the item removed. Records the change.

POST `/api/items/{id}/restore`

requires `items.update`

Brings an archived item back to active. Records the change.

DELETE `/api/items/{id}`

requires `items.delete`

Permanent delete. Only valid once an item is already archived — reject otherwise. Not logged.

POST `/api/items/{id}/move`

requires `items.update`

The "Move / Use Item" action — see the full flow in Section 5, including the quantity-split behavior.

```
// request
{ "quantity": 4, "storageId": "ST-1002", "status": "in_use" }

// response — all of the quantity moved: the item updated in place
{ "updated": { "id": "IT-1013", "storageId": "ST-1002", "status": "in_use" } }

// response — part of the quantity moved: a new item is created for that part
{ "updated": { "id": "IT-1013", "quantity": 2 },
  "created": { "id": "IT-1101", "quantity": 4, "storageId": "ST-1002",
               "status": "in_use", "name": "Beaker Set", "tags": [] } }
```

POST `/api/items/reassign`

requires `items.update`

The "pick an existing item while adding to a storage" flow — see Section 5.

```
{ "itemId": "IT-1013", "storageId": "ST-1002" }
```

POST `/api/items/import`

requires `items.create`

File upload (CSV or spreadsheet) — see the full import flow in Section 5.

```
// response
{ "imported": 18, "skipped": 2, "errors": [
  { "row": 5, "reasons": ["Unknown storage \"ST-9999\""] } ] }
```

3.5 Users

GET `/api/users?search=&roleId=&sort=&page=`

requires `users.read`

POST `/api/users`

requires `users.create`

```
{ "name": "Jane Doe", "username": "jane", "email": "jane@x.com", "phone": "",
  "status": "active", "roleId": "RL-1002" }
// email is required — 400 if missing
// 409 if username already taken (case-insensitive)
```

PATCH `/api/users/{id}`

requires `users.update`

POST `/api/users/{id}/invite`

requires `users.update`

The "Send Invite" / table reset-password action — see the Invite / Reset Password flow in Section 5. 400 if the user has no email. The frontend never calls anything beyond this: it shows a confirmation and a toast, nothing more. Everything past that point (generating a real token, emailing a link, and a page that consumes it to set the password) is server-side-only and not implied by any existing client call — see Section 6.

DELETE `/api/users/{id}`

requires `users.delete`

Unconditional today — see Section 7.

3.6 Roles

GET `/api/roles?search=`

requires `roles.read`

```
[ { "id": "RL-1002", "name": "Staff", "description": "...",
    "permissions": { "storage": {"create":true,"read":true,"update":true,"delete":false}, ... },
    "createdAt": 1733900000000, "userCount": 3 } ]
```

POST `/api/roles`

requires `roles.create`

PATCH `/api/roles/{id}`

requires `roles.update`

DELETE `/api/roles/{id}`

requires `roles.delete`

409 Conflict if any user still holds this role, with the count in the error body ("Reassign its N user(s) to another role first").

3.7 Item Log

GET `/api/item-log?search=&type=&rackId=&from=&to=&sort=&page=`

requires read access, gated the same as any other page — map to `items.read` unless a dedicated module is added

No create/update/delete is exposed to clients — rows only ever appear as a side effect of Item operations (Section 5). `from` / `to` are inclusive date bounds.

3.8 Dashboard

GET `/api/dashboard/summary`

```
{ "totalStorageUnits": 23, "totalLocations": 4, "totalItems": 311, "typesInUse": 6 }
```

GET `/api/storage-units?itemCount=0`

"Needs Attention" list — reuses the Storage Units list endpoint rather than a separate one.

GET `/api/storage-units?sort=createdAt-desc&pageSize=5`

"Recently Added" list — same reuse.

4. Pages → API Call Map

Every page, and the exact endpoints (Section 3) it calls, in the order it typically calls them.

> The frontend now ships two views — `/web` (sidebar) and `/mobile` (bottom tab bar) — each with its own copy of every page below except Scan QR, which is mobile-only. Both copies of a given page call the identical endpoints listed here; the split is presentation-only; it has no effect on the API surface.

index.html — Login

`js/login.js`

1. Submit credentials → `POST /api/auth/login`
2. "Forgot password" link → `POST /api/auth/forgot-password/request`, then `POST /api/auth/forgot-password/confirm`

dashboard.html — Overview

`js/overview.js`

1. On load → `GET /api/dashboard/summary`
2. "Needs Attention" panel → `GET /api/storage-units?itemCount=0`
3. "Recently Added" panel → `GET /api/storage-units?sort=createdAt-desc&pageSize=5`

items.html — All Items

`js/items.js`

1. On load / any filter, search, sort, or page change → `GET /api/items` with the matching query params
2. "View Archive" toggle → same endpoint with `archived=true`
3. Add Item submit → `POST /api/items`
4. Edit Item submit → `PATCH /api/items/{id}`
5. Status-picker → `POST /api/items/{id}/move`
6. Archive button → `POST /api/items/{id}/archive`
7. Restore / Delete-forever buttons (Archive view) → `POST /api/items/{id}/restore` or `DELETE /api/items/{id}`
8. Import flow → `POST /api/items/import`
9. CSV/PDF export — stays client-side, no endpoint required

storage.html — Storage Units

`js/storage.js`

1. On load / filter / search / sort / page change → `GET /api/storage-units`
2. Add Storage submit → `POST /api/storage-units`
3. Edit button (card) → `PATCH /api/storage-units/{id}`
4. Delete button (card) → `DELETE /api/storage-units/{id}`

storage-detail.html — Single Storage

`js/storage-detail.js`

1. On load → `GET /api/storage-units/{id}`, then `GET /api/items?storageId={id}` for the Items panel
2. Click-to-edit name / location / tags / pictures → `PATCH /api/storage-units/{id}` with just that one field
3. Items panel inline row edit → `PATCH /api/items/{id}`
4. Items panel status-picker → `POST /api/items/{id}/move`
5. Items panel archive button → `POST /api/items/{id}/archive`
6. "+ Add Item" → `POST /api/items` (storage preset to this unit)
7. "Delete Storage" → `DELETE /api/storage-units/{id}`

item-detail.html — Single Item

`js/item-detail.js`

1. On load → `GET /api/items/{id}`, then `GET /api/item-log?itemName={name}` for the Activity panel
2. Click-to-edit name / storage / tags / quantity / remarks / pictures → `PATCH /api/items/{id}` with just that one field
3. Status-picker → `POST /api/items/{id}/move`
4. "Remove to Archive" → `POST /api/items/{id}/archive`
5. "Restore" / "Delete Forever" (archived view) → `POST /api/items/{id}/restore` or `DELETE /api/items/{id}`

mobile/scan.html — Scan QR (mobile only, no web equivalent)

`js/scan.js`

1. On successful scan or manual ID entry → `GET /api/storage-units/{id}` to confirm the storage exists, then redirect to the Storage Detail screen for it (see the Scan QR flow in Section 5)
2. Everything else — requesting the camera, decoding frames, parsing the ID out of the scanned code — happens entirely on-device; no other endpoint is involved

log.html — IN/OUT Log

`js/log.js`

1. On load / filter / search / sort / page change → `GET /api/item-log` with the matching query params
2. CSV/PDF export — client-side, no endpoint required

storerooms.html — Locations

`js/storerooms.js`

1. On load / search / sort / page change → `GET /api/locations`
2. Add/Edit submit → `POST /api/locations` or `PATCH /api/locations/{id}`
3. Delete button → `DELETE /api/locations/{id}`

users.html — Users

`js/users.js`

1. On load / search / role filter / sort / page change → `GET /api/users`
2. Add/Edit submit → `POST /api/users` or `PATCH /api/users/{id}`
3. "Send Invite" (Add/Edit form) or the table's key-icon action → `POST /api/users/{id}/invite` (Add/Edit also calls `POST /api/users` first if the form was still creating a new user)
4. Delete button → `DELETE /api/users/{id}`

roles.html — Roles & Permissions

`js/roles.js`

1. On load / search / sort / page change → `GET /api/roles`
2. Add/Edit submit (name, description, full permission matrix) → `POST /api/roles` or `PATCH /api/roles/{id}`
3. Delete button → `DELETE /api/roles/{id}`

5. Key Process Flows

Step-by-step behavior for every multi-step or branching action in the app. These are the flows the backend must reproduce exactly — not just the data shape, but the order of checks and side effects.

Login

Trigger: user submits the sign-in form.

1. Check the submitted username and password against stored credentials.
2. If they don't match, show an error and shake the form — do not reveal which of the two was wrong.
3. If they match, start a session, record who is signed in, and send the user to the Dashboard.

Forgot Password

Trigger: user clicks "Forgot password" on the login screen.

1. User enters their username.
2. If no account matches, show an error and stop here.
3. If an account matches, generate a one-time verification code and email it to the account's address. **The code must never be returned to the client** — only delivered by email.
4. User enters the code plus a new password (and confirmation).
5. Reject if: the code doesn't match, the new password is under 6 characters, or the confirmation doesn't match the new password.
6. Otherwise, update the stored password and return the user to the login form with a success message.

Invite / Reset Password (Users page)

Trigger: an admin clicks "Send Invite" on the Add/Edit User form, or the key-icon action on a row in the Users table.

1. If the user has no email on file, stop here: redirect to the Edit User form instead, with a message explaining that an email is required first. Email is mandatory on every Add/Edit User submission for exactly this reason.
2. If triggered from "Send Invite" while still creating a new user, validate and save the user record first (same validation as a normal Save), so there is a real record for the invite to reference.
3. Ask the admin to confirm: "Send a password reset link to [name]?"
4. On confirmation, send the email and show the admin a single "Invitation sent to [email]" acknowledgement. Nothing else happens on the admin's screen — no password field is ever shown here, and the admin never sees or sets the user's password on their behalf.
5. Setting the actual password happens only on the recipient's end, by following the link in that email — a step this app's frontend does not implement further (there is no real email service, and no page that consumes the link); it only triggers the request.

> The backend needs to actually implement the part the frontend only gestures at: generate a real, single-use reset token, email a link containing it, and serve a page/endpoint that lets the recipient set a new (hashed) password when they follow it — mirroring the two ends of the existing Forgot Password flow above, but reachable by an admin inviting someone else rather than only by the account holder.

Scan QR (mobile-facing entry point)

Trigger: user opens the Scan QR screen and either grants camera access or types an ID directly.

1. If the camera isn't available (no permission, unsupported browser, or not a secure connection), skip straight to the manual ID field below — it's always present as a fallback, not just an error state.
2. Otherwise, continuously read frames from the device's rear-facing camera and attempt to decode a QR code from each one.
3. Once a code decodes, extract the storage ID from it: the app's own QR codes encode a full link to that storage's detail screen (e.g. ".../storage-detail.html?id=ST-1008"), so pull the ID out of that link's query string. If the decoded text isn't a link at all, treat it as a bare ID directly.
4. Look the ID up. If it doesn't match a real storage unit, show "No storage found for [code]" and keep scanning — don't get stuck on a bad read, and don't repeat the same error message on every frame while the camera keeps seeing the same bad code.
5. If it matches, stop scanning immediately (so a lingering camera view can't double-trigger), show a brief success state, and open that storage's full detail screen.
6. The manual ID field runs through the exact same lookup-and-open logic as a successful scan — it isn't a lesser fallback, just a different way to supply the same ID.
7. Release the camera (stop the video stream) whenever the user navigates away, backgrounds the tab, or explicitly stops scanning — never leave the camera running after the user has left the screen.

> This is the core "point your phone at a shelf and see what's in it" flow. Nothing about it requires a dedicated endpoint beyond the existing Storage Unit lookup (Section 3.3) — the scanning and decoding happen entirely on the device.

Add Item (new item vs. existing item)

Trigger: user opens "Add Item" and types a name.

1. As the user types the item name, search existing active (non-archived) items by name, ID, tag, remarks, or current storage, and show matches below the field.
2. User now has two paths:

PATH A User ignores the suggestions and fills in the rest of the form (quantity, storage, tags, etc.)

1. On submit, create a brand-new item with the given fields.
2. Record an IN movement for the full quantity against the chosen storage.

PATH B User selects one of the suggested existing items (by click or by pressing Enter while a suggestion is highlighted)

1. A Storage must already be selected in the form first — if not, show an error and stop.
2. Ask the user to confirm: "Move [item] into [storage]? It will be taken out of [current location]."
3. On confirmation, change only that item's storage to the one selected in the form. Nothing else about the item changes, and no new item is created.

> Either path must be fully supported — pressing Enter with a suggestion highlighted must never create a duplicate item by accident.

Move / Use Item (the most important flow in the app)

Trigger: user picks a status badge/button on an item (from the Items table, a storage's Items panel, or the item's own page) to relocate it, change its status, or both.

1. User chooses a quantity (from 1 up to the item's current quantity), a destination storage, and a destination status.
2. Reject if the quantity is outside that range, or if nothing would actually change (same quantity, same storage, same status as now).
3. Branch on how much of the item is being moved:

ALL OF IT the chosen quantity equals the item's current quantity

1. Update that same item's storage and status in place. Nothing is split or duplicated.

PART OF IT the chosen quantity is less than the item's current quantity

1. Reduce the original item's quantity by the amount being moved. The original item stays where it was.
2. Create a brand-new item record — a clone of the original (same name, tags, remarks, pictures) — with the moved quantity, the destination storage, and the destination status, and a newly generated ID.
4. If the storage changed (either branch): record an OUT movement against the source storage and an IN movement against the destination storage, both for the quantity that moved.
5. If the status changed (either branch): record a status-change movement for the affected item and quantity.
6. Return whichever item now represents the moved portion (the original if nothing split, or the newly created one if it did) plus the original if it was only partially reduced.

> Every field except ID, quantity, storage, and status is copied verbatim onto a split-off item — it is a full clone of the original's attributes. The original item always keeps its own ID; a split-off portion always gets a new one.

Archive / Restore / Permanently Delete an Item

Trigger: user clicks Archive, Restore, or Delete Forever on an item.

1. **Archive** — ask for confirmation ("Remove [item] to the archive?"). On confirmation, mark the item removed and record the change. The item is not deleted — it moves to the Archive view.
2. **Restore** — no confirmation needed. Mark the item active again ("in store") and record the change.
3. **Permanently Delete** — only ever available from the Archive view (an item must already be archived). Ask for confirmation ("Permanently delete [item]? This cannot be undone."). On confirmation, remove the record entirely. This action is not recorded in the movement log.

Add Storage (with optional existing items)

Trigger: user submits the "Add Storage" form.

1. Required: a unique Storage ID, a name, a type, and a location. Reject if the ID is already in use (case-insensitive).
2. Optional: tags, pictures, and a list of existing items to move into this storage as soon as it's created (searched the same way as in the Add Item flow, but allowing multiple picks via removable chips instead of one pick).
3. Create the storage unit.
4. If any existing items were picked, reassign each one's storage to the newly created unit. This does not go through the Move flow — it's a direct reassignment, and does not trigger an IN/OUT movement record in the current app.

Bulk Import Items (CSV / spreadsheet)

Trigger: user uploads a file on the Items page.

1. Parse the file into rows with columns: name, item ID, tags, quantity, storage ID, remarks, status.
2. Validate every row independently:

1. Name is required.
2. Quantity must be a number of at least 1.
3. If a storage ID is given, it must match an existing storage unit.
4. If an item ID is given, it must not already exist, and must not be repeated elsewhere in the same file.

3. Show the user a preview: how many rows are valid, how many have errors, and what each error is.
4. On confirmation, create only the valid rows (invalid rows are skipped, not fixed) — each one records an IN movement.
5. Report back how many were imported and how many were skipped.

> A file with some bad rows must still successfully import the good ones — never reject the whole batch over a handful of bad rows.

Delete Safeguards (Location, Role)

Trigger: user clicks Delete on a Location or a Role.

1. **Location** — before even asking for confirmation, check whether any storage unit still belongs to it. If so, show an error naming how many ("Move or delete its N storage unit(s) first") and stop — do not ask to confirm. Only ask for delete confirmation once the count is zero.
2. **Role** — same pattern: check whether any user still holds this role first; if so, show an error ("Reassign its N user(s) to another role first") and stop; only proceed to a confirmation step once the count is zero.

> The backend must enforce both checks itself as well — the frontend's pre-check is a convenience, not a substitute for a server-side guarantee.

6. Authentication & Authorization

The current implementation is a stub, but a more complete one than it first looks: the User record now carries a `password` field and the Users page has a "Send Invite" / reset action, so a password *can* exist and gate login — but nothing in the frontend ever actually writes that field. The invite action only confirms and shows a toast; the password is modeled as being set entirely on the recipient's end, by a link in an email this demo never actually sends or serves a page for. The signed-in username is still held only in browser sessionStorage, and the one real credential-entry surface remains the Forgot Password flow described in Section 5 (scoped to the single hardcoded `admin` account, not to invited users).

> However, the authorization model itself is fully designed in the frontend and used throughout the UI — it is worth preserving exactly:

- Each role carries a per-module create/read/update/delete permission flag, for modules: storage, items, locations, users, roles.
- There is no per-user row-level or project-level scoping anywhere in this app — any user with read permission on a module sees every row in that module, with no further restriction.

What needs to be built for real:

- A genuine, properly hashed password field on the User record — the frontend's `password` is a plaintext demo value with no real way to populate it, so it establishes the shape (one password per user, gating login) but the value itself is not fit to migrate as-is.
- A real invite/reset endpoint that actually generates a token, emails a link, and serves a page that lets the recipient set that password — the frontend's "Send Invite" only confirms and shows a toast; see the Invite / Reset Password flow in Section 5 for exactly what's missing.
- A real login endpoint issuing a session or token, replacing the single hardcoded demo credential.
- Server-side enforcement of every endpoint in Section 3 against each user's role permissions — today permission checks are purely a UI-hiding concern; nothing stops a motivated client from calling the underlying operations directly, since everything runs in the browser with no server to check against.

7. Integrity Requirements

- Every Item operation that changes quantity, storage, or status must record its movement log entry(ies) as part of the same operation — both happen, or neither does.
- **Role deletion** must be blocked while any user still holds that role. There is no "system/seeded role" protection in this app — the in-use check is the only guard needed.
- **Location deletion** must be blocked while any storage unit still belongs to it.
- **Storage Unit deletion is a real gap in the app today**: the current frontend deletes a storage unit unconditionally, with no check against and no reassignment of items still pointing at it — those items are left referencing a storage that no longer exists. This is not a deliberate design, just something the client-only app never had to handle; the backend needs an explicit decision (block the delete while items remain assigned, or automatically unassign them).
- **User deletion** is currently unconditional, with no reference checks (a deleted user may still be named in historical movement log entries — that's fine, since the log only ever stores a snapshot of the username, not a live reference).
- **Item permanent delete** is only valid once an item is already archived.
- The movement log is append-only — nothing in the app ever edits or deletes an existing log entry.
- Username uniqueness (Users) and Storage/Item ID uniqueness are checked case-insensitively, matching the app's exact behavior today.